
CS683 Programming Assignment -2 report

Submitted by: 24B1015, 24B0995, 24B1021, 24B1037

1 Task 1: Stride Pattern Prefetching

1.1 Task 1A: Understanding the IP-stride prefetcher

1.1.1 How the prefetcher works

The IP-stride prefetcher (`prefetcher/ip_stride.l1d_pref`) sits at the L1 data cache. Its idea is that a single load or store instruction, identified by its instruction pointer (IP), tends to walk through memory with a constant stride (for example, a loop over an array). It remembers the last address each IP touched, computes the stride between consecutive accesses of that IP, and once the same stride is seen twice in a row it prefetches the next few addresses along that stride. The code is walked through below, section by section; the line numbers in the listings are the line numbers in the source file.

1. Parameters and the tracker entry (lines 4–30).

```

4  #define IP_TRACKER_COUNT 24
5  #define PREFETCH_DEGREE 3
6  #define MASK_IP 0xFF
7
8  class IP_TRACKER {
9  public:
10     // the IP we're tracking
11     uint64_t ip;
12
13     // the last address accessed by this IP
14     uint64_t last_cl_addr;
15
16     // the stride between the last two addresses accessed by this IP
17     int64_t last_stride;
18
19     // use LRU to evict old IP trackers
20     uint32_t lru;
21
22     IP_TRACKER () {
23         ip = 0;
24         last_cl_addr = 0;
25         last_stride = 0;
26         lru = 0;
27     };
28 };
29
30 IP_TRACKER trackers[IP_TRACKER_COUNT];

```

- `IP_TRACKER_COUNT = 24`: the prefetching table has only 24 entries.
- `PREFETCH_DEGREE = 3`: when the prefetcher fires it issues 3 prefetches. The degree is fixed and never changes.
- `MASK_IP = 0xFF`: only the low 8 bits of the IP are used to identify an instruction. Different instructions whose IPs share the low 8 bits therefore share one tracker (aliasing).
- Each `IP_TRACKER` entry stores: the (masked) IP it belongs to; the cache-line address of the last access by that IP (`last_cl_addr`); the stride observed between its last two accesses (`last_stride`, signed, in cache lines); and an LRU rank used to choose which entry to replace when the table is full.

- The table `trackers[]` is a single global array, so it is shared by everything that uses this prefetcher (fine for the single-core configuration used here).

2. Initialisation (lines 33–39).

```

33 void CACHE::l1d_prefetcher_initialize()
34 {
35     cout << "CPU " << cpu << " L1D IP-based stride prefetcher" << endl;
36     for (int i=0; i<IP_TRACKER_COUNT; i++)
37         trackers[i].lru = i;
38
39 }
```

This function is called once when the simulation starts. It prints a banner and gives the 24 entries the distinct LRU ranks 0,1,...,23. Rank 0 means most recently used and rank 23 (IP_TRACKER_COUNT-1) means least recently used, so at any time the ranks form a permutation of 0...23 and exactly one entry is the replacement victim. All entries start with `ip = 0`, `last_cl_addr = 0` and `last_stride = 0`.

3. Entry point: cache-line address and IP (lines 41–46).

```

41 void CACHE::l1d_prefetcher_operate(uint64_t addr, uint64_t ip, uint8_t cache_hit, uint8_t
    type, uint8_t critical_ip_flag)
42 {
43     // check for a tracker hit
44     uint64_t cl_addr = addr >> LOG2_BLOCK_SIZE;
45
46     ip = ip & MASK_IP;
```

`l1d_prefetcher_operate` is called by the L1D for each access it handles. The prefetcher works on cache lines, not bytes: shifting the byte address right by `LOG2_BLOCK_SIZE` drops the offset inside the 64-byte line, giving `cl_addr`. The IP is reduced to its low 8 bits. The arguments `cache_hit` and `type` are not used in this file, so the prefetcher trains on every access it is called for, whether it hit or missed in the L1D.

4. Looking up the tracker (lines 48–52).

```

48     int index = -1;
49     for (index=0; index<IP_TRACKER_COUNT; index++) {
50         if (trackers[index].ip == ip)
51             break;
52     }
```

The table is fully associative: a linear search compares the masked IP against every entry. If an entry matches, the loop breaks with `index` pointing at it. If none matches, the loop runs to the end and `index` equals `IP_TRACKER_COUNT`.

5. A new IP: allocating a tracker (lines 54–74).

```

54 // this is a new IP that doesn't have a tracker yet, so allocate one
55 if (index == IP_TRACKER_COUNT) {
56
57     for (index=0; index<IP_TRACKER_COUNT; index++) {
58         if (trackers[index].lru == (IP_TRACKER_COUNT-1))
59             break;
60     }
61
62     trackers[index].ip = ip;
63     trackers[index].last_cl_addr = cl_addr;
64     trackers[index].last_stride = 0;
65
66     //cout << "[IP_STRIDE] MISS index: " << index << " lru: " << trackers[index].lru
67     << " ip: " << hex << ip << " cl_addr: " << cl_addr << dec << endl;
68     for (int i=0; i<IP_TRACKER_COUNT; i++) {
69         if (trackers[i].lru < trackers[index].lru)
70             trackers[i].lru++;
71     }
72     trackers[index].lru = 0;
73     return; //metadata_in;
74 }

```

- **Choosing the victim (57–60).** The entry whose LRU rank is 23 (the least recently used) is selected for replacement.
- **Initialising it (62–64).** The entry is overwritten with the new IP and the current line address. `last_stride` is set to 0 because no stride is known yet.
- **LRU update (67–71).** Every entry that was more recently used than the victim (rank smaller than the victim's) moves one step towards “old” (rank +1), and the new entry becomes rank 0. Since the victim had rank 23, all other entries are incremented and the ranks remain a permutation.
- **No prefetch (73).** With only one address seen, there is no stride to compute, so the function returns without prefetching.

6. Sanity check and stride calculation (lines 76–90).

```

76 // sanity check
77 // at this point we should know a matching tracker index
78 if (index == -1)
79     assert(0);
80
81 // calculate the stride between the current address and the last address
82 // this bit appears overly complicated because we're calculating
83 // differences between unsigned address variables
84 int64_t stride = 0;
85 if (cl_addr > trackers[index].last_cl_addr)
86     stride = cl_addr - trackers[index].last_cl_addr;
87 else {
88     stride = trackers[index].last_cl_addr - cl_addr;
89     stride *= -1;
90 }

```

When execution reaches here, a tracker for this IP exists. The `assert` guards against an impossible state. The stride is the difference between the current and the previous cache-line

address of this IP, in units of cache lines. Because the addresses are unsigned, the code subtracts the smaller from the larger and negates the result if the access moved backwards, so a descending access pattern gives a negative stride.

7. Ignoring repeated lines (lines 94–96).

```

94      // don't do anything if we somehow saw the same address twice in a row
95      if (stride == 0)
96          return; //metadata_in;

```

A stride of 0 means the same cache line was accessed again (for example several accesses to the same line). There is nothing to learn or prefetch from that, so the function returns early. Note that this early return does *not* update `last_cl_addr`, `last_stride` or the LRU ranks.

8. The trigger and the prefetch loop (lines 98–117).

```

98      // only do any prefetching if there's a pattern of seeing the same
99      // stride more than once
100     if (stride == trackers[index].last_stride) {
101
102         // do some prefetching
103         for (int i=0; i<PREFETCH_DEGREE; i++) {
104             uint64_t pf_address = (cl_addr + (stride*(i+1))) << LOG2_BLOCK_SIZE;
105
106             // only issue a prefetch if the prefetch address is in the same 4 KB page
107             // as the current demand access address
108             if ((pf_address >> LOG2_PAGE_SIZE) != (addr >> LOG2_PAGE_SIZE))
109                 break;
110
111             // check the MSHR occupancy to decide if we're going to prefetch to the L1 or
112             L2
113             if (MSHR.occupancy < (MSHR.SIZE>>1))
114                 prefetch_line(ip, addr, pf_address, FILL_L1, 0);
115             else
116                 prefetch_line(ip, addr, pf_address, FILL_L2, 0);
117         }
118     }

```

- **Trigger (100).** Prefetching only happens if the new stride equals the stored `last_stride`, i.e. the same stride has been seen twice in a row. This is the prefetcher's only form of confidence: one repetition. Since a new entry starts with `last_stride = 0`, an IP needs three accesses before its first prefetch (the first allocates the entry, the second records the stride, the third confirms it).
- **Addresses (103–104).** For $i = 0, 1, 2$ it prefetches the line at `cl_addr + stride · (i + 1)`, converted back to a byte address, so with stride s it prefetches the next three lines along the pattern.
- **Page boundary (108–109).** The prefetch address must be in the same 4 KB page as the demand access (the usual reason is that consecutive pages are not necessarily contiguous or even mapped, so a stride run cannot safely be followed across a page. Also the prefetcher does not know whether the next page is actually valid/mapped.). On the first violation the loop stops, since the later prefetches in the same direction would also be outside the page.
- **Fill level (112–115).** `prefetch_line` hands the request to the cache. If fewer than half of the L1D MSHR entries are in use the line is brought into the L1D (`FILL_L1`); otherwise it is only brought into the L2 (`FILL_L2`), so that prefetches do not use up the miss-handling resources that demand misses need.

9. Updating the entry (lines 119–128).

```

119     trackers[index].last_cl_addr = cl_addr;
120     trackers[index].last_stride = stride;
121
122     for (int i=0; i<IP_TRACKER_COUNT; i++) {
123         if (trackers[i].lru < trackers[index].lru)
124             trackers[i].lru++;
125     }
126     trackers[index].lru = 0;
127     return; //metadata_in;
128 }

```

Whether or not a prefetch was issued, the entry now remembers the current line address and the current stride, ready for the next access of this IP. The entry then becomes the most recently used: every entry that was more recent than it is aged by one and it gets rank 0. Note that only the *last* stride is kept as history; one irregular access overwrites it.

10. Fill hook and final statistics (lines 130–139).

```

130 void CACHE::l1d_prefetcher_cache_fill(uint64_t v_addr, uint64_t addr, uint32_t set,
131                                       uint32_t way, uint8_t prefetch, uint64_t v_evicted_addr, uint64_t evicted_addr,
132                                       uint32_t metadata_in)
133 {
134     return; //metadata_in;
135 }
136
137 void CACHE::l1d_prefetcher_final_stats()
138 {
139     cout << "CPU " << cpu << " L1D PC-based stride prefetcher final stats" << endl;
140     cout << "Degree: " << PREFETCH_DEGREE << endl;
141 }

```

`l1d_prefetcher_cache_fill` is called whenever a line is filled into the L1D. This prefetcher does not use it (the body just returns). `l1d_prefetcher_final_stats` prints the degree at the end of the simulation. There is no accuracy tracking in this prefetcher; the accuracy reported in the results comes from ChampSim's own prefetch counters.

1.1.2 Limitations

The IP-stride prefetcher only works for accesses with one constant stride per IP. Its main limitations, which motivate SPP, are:

- **No tolerance to irregularity.** Each entry stores a single stride and a single previous address, and a prefetch is issued only when $s_n = s_{n-1}$, where $s_n = A_n - A_{n-1}$. One irregular access overwrites `last_stride`, so the next access cannot match it and the stride has to be re-learned and re-confirmed. A single irregular access therefore suppresses prefetching for about two subsequent accesses. There is no confidence counter that could absorb such noise.
- **Cannot learn alternating or periodic strides.** Patterns whose stride sequence is periodic but not constant, such as $\{+1, +3, +1, +3, \dots\}$, never produce two equal consecutive strides, so no prefetch is ever issued even though the pattern is fully predictable. Typical sources are structure-of-fields traversals, padded 2D arrays and nested loops. Only one delta of history is kept, so such patterns cannot be captured.
- **No support for irregular or indirect accesses.** Pointer chasing and indirect accesses such as `A[B[i]]` have no constant stride, so the prefetcher gives no benefit and only trains on noise.
- **Training delay.** An IP needs three accesses before its first prefetch, which hurts short loops and short streams.
- **Page boundary.** Prefetching stops at the 4KB page boundary and each new page has to be trained again, so the lookahead is truncated near the end of every page.
- **Fixed degree and no feedback.** The degree is fixed and is not adapted to accuracy or timeliness, so the prefetcher may be too aggressive on irregular code (cache pollution, wasted bandwidth) or too shallow on regular code.
- **Per-IP table pressure.** Training state is kept per IP in a small LRU table. Programs with many load IPs thrash it, and patterns learned by one IP cannot be shared with other IPs that behave identically.

SPP addresses these by keeping a history of several recent deltas (a signature) and by predicting the next delta from that history, so alternating patterns can be captured and a single irregular access does not destroy the learned state.

1.1.3 Results

All runs use 25M warmup and 25M simulated instructions, with LRU at the L2C. The baseline is the same configuration with no L1D prefetcher. Speedup is $IPC_{IP\text{-}stride}/IPC_{baseline}$, and coverage is $(L1D \text{ load misses}_{base} - L1D \text{ load misses}_{IP\text{-}stride})/L1D \text{ load misses}_{base}$. Accuracy is the **ACCURACY** value printed by ChampSim for the L1D, which is the number of useful prefetches divided by the number of prefetches *issued to the lower level* (the prefetches that missed in the L1D and went to the L2), not by all prefetches requested by the prefetcher.

Trace	Prefetcher	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage (%)
Trace1-001	Baseline	0.2323	1.000	10.68	–	–
	IP-stride	0.2333	1.005	10.12	53.0	5.3
Trace2-002	Baseline	0.2506	1.000	13.54	–	–
	IP-stride	0.2505	1.000	13.44	38.2	0.7
Trace3-001	Baseline	0.3820	1.000	3.40	–	–
	IP-stride	0.3829	1.002	3.32	16.0	2.5

Table 1: Baseline versus IP-stride (degree 3) on the three traces.

Table 1 shows that IP-stride is almost neutral on these traces. Table 2 gives the prefetch counters behind the accuracy and coverage numbers.

Trace	Queued	Hit in L1D	To lower level	Useful	Useless	Late
Trace1-001	155 922	102 542	30 956	16 408	8 834	2 807
Trace2-002	78 701	57 796	11 426	4 361	6 300	272
Trace3-001	103 500	70 396	17 007	2 713	10 595	3 067

Table 2: L1D prefetch counters of IP-stride. “Queued” is the number of prefetch requests put into the L1D prefetch queue (the L1D PQ ACCESS line). “Hit in L1D” is the number of queued prefetches whose line was already in the L1D. “To lower level” is the denominator ChampSim uses for accuracy. No prefetch crosses a page boundary.

Observations.

- **Performance gain is negligible.** The speedups are 1.005, 1.000 and 1.002, so IP-stride changes IPC by at most 0.5%.
- **Coverage is low**, only 5.3%, 0.7% and 2.5% of the baseline L1D load misses are removed. Trace2 gains almost nothing, and it is also the trace with the highest baseline L1D load MPKI (13.5), so there is plenty of room that the prefetcher does not use.
- **Accuracy varies a lot between traces:** 53.0%, 38.2% and 16.0%. In Trace3 the useless prefetches (10 595) are almost four times the useful ones (2 713), and the late prefetches (3 067) outnumber the useful ones, so many prefetches also arrive after the demand access.
- **Most prefetches never leave the L1D.** Of the prefetches that look up the L1D, 80.2%,

84.4% and 84.1% (Trace1, 2, 3) find the line already present (the L1D PREFETCH hit rate), so they fetch nothing. Only 19.9%, 14.5% and 16.4% of the queued prefetches go to the L2 (“To lower level” over “Queued”). The rest hit in the L1D or merge with a request already in the queue (about 8% of the queued prefetches). The prefetcher therefore re-requests the same lines many times: lookups with no benefit.

- **ChampSim’s “ISSUED” counts every prefetch twice.** In this build, `pf_issued` is incremented once when the prefetch enters the queue and again when its translation request is sent to the STLB, so `ISSUED = PQ ACCESS + PQ TO_CACHE` exactly in all three traces (for Trace1, $155\,922 + 142\,926 = 298\,848$). We therefore report the number of queued prefetches instead of “ISSUED”.
- **Page-boundary rule works.** There are zero cross-page prefetches in all traces, as the code intends.
- **Trace3 has the least to gain:** its baseline L1D load MPKI is already low (3.40), so even a perfect prefetcher could remove only a small number of misses.

These results are the reference point for the SPP evaluation.

1.2 Task 1B: Stride Pattern Prefetcher (SPP)

1.2.1 Design

The SPP prefetcher keeps the per-IP organisation of the IP-stride prefetcher, but every IP entry holds two predictors instead of one: a cheap *uniform* predictor for constant strides and a small *signature table* for strides that follow a repeating pattern. A confidence counter in each predictor is raised on correct predictions and lowered on wrong ones, so a single irregular access no longer destroys what was learned. On top of this, a small prefetch filter measures how many prefetches are useful, and the prefetch degree is throttled from that measurement.

Overview.

Figure 1 shows the structures. Only demand loads (`type == LOAD`) are used for training and prediction. Strides are always measured in cache lines.

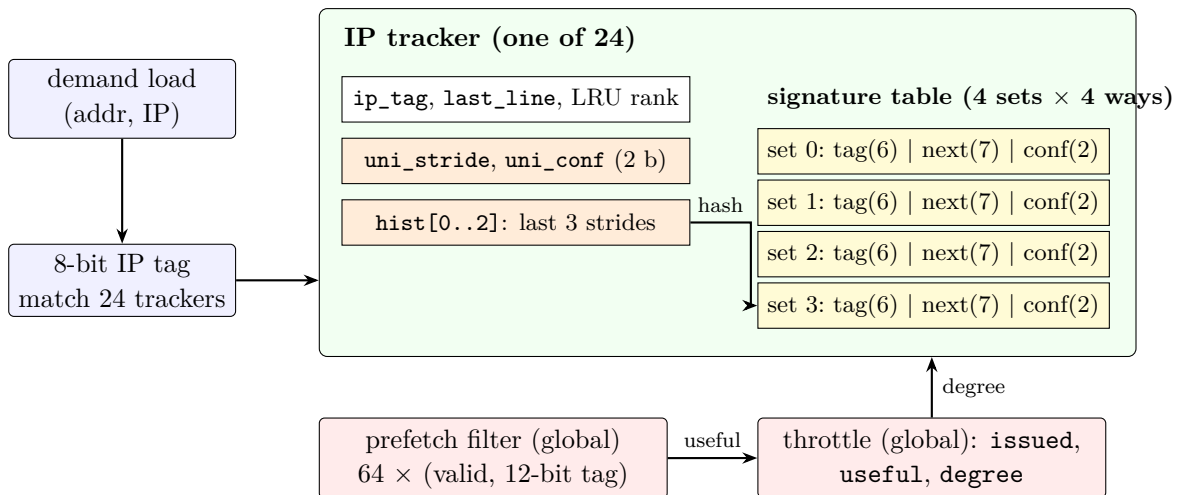


Figure 1: Structures of the SPP prefetcher.

IP tracker.

Like IP-stride, the trackers are a plain array of 24 entries, indexed by an 8-bit IP tag and replaced in LRU order. Besides the IP tag and the last accessed cache line, each tracker holds:

- a **uniform predictor**: a stride `uni_stride` and a 2-bit confidence `uni_conf`;
- a **stride history** `hist[3]` with the last three strides of this IP;
- a private **signature table** of $4 \text{ sets} \times 4 \text{ ways}$. Each entry holds a 6-bit tag, the stride that followed the signature (`next`, 7 bits signed) and a 2-bit confidence. A confidence of 0 marks an invalid entry.

The *signature* of an IP is the hash of its three-stride history (FNV-style mix of the three strides); two bits of it select the set and six bits form the tag. A signature therefore stands for “the last three strides were a, b, c ” and the table learns which stride d usually comes next.

Training.

On every demand load of a known IP the stride $s = \text{line} - \text{last_line}$ is computed. A stride of 0 (same line) is ignored. A stride larger than ± 63 lines is treated as a jump: the uniform predictor and the history are cleared and no learning takes place. Otherwise both predictors are trained:

- **Uniform predictor.** If $s = \text{uni_stride}$, `uni_conf` is increased (saturating at 3). Otherwise it is decreased by 1 (the miss penalty), and only once it has reached 0 is `uni_stride` replaced by s . A constant-stride stream therefore needs three accesses to reach the usable confidence of 2, but one stray stride only costs one confidence step instead of the whole entry.
- **Signature table.** The signature of the *old* history is looked up. On a hit whose `next` equals s the confidence is increased (saturating at 3), otherwise it is decreased and the entry is cleared when it reaches 0. On a miss the weakest way of the set is chosen: if its confidence is 0 it is replaced by a new entry with (tag, s , 1), otherwise its confidence is only decreased. Stale or strong entries are thus replaced gradually and not at the first conflict.

Finally s is shifted into `hist` and `last_line` is updated.

Uniform / non-uniform classification and fallback.

An IP is classified as *uniform* when $\text{uni_conf} \geq 2$, i.e. its recent strides have mostly been the same. Otherwise it is *non-uniform* and the signature table is used. This is a fallback chain (Figure 2): constant strides are handled by the cheap uniform predictor, which needs no history, and only IPs that are not uniform are handed to the signature table, which can also follow alternating or otherwise periodic strides. If the signature table has no confident entry for the current history, nothing is prefetched.

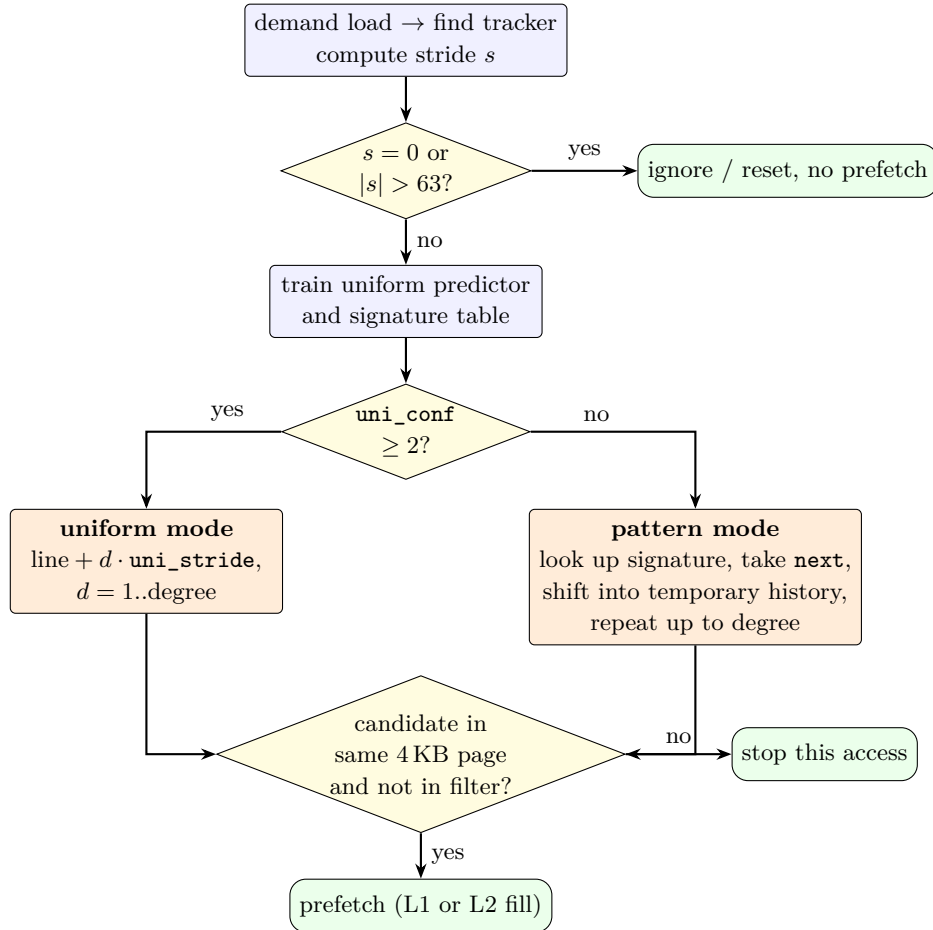


Figure 2: Decision flow of the SPP prefetcher for one demand load.

Prefetch generation, degree and page limit.

In uniform mode up to **degree** lines $\text{line} + d \cdot \text{uni_stride}$ are requested. In pattern mode the predictor walks ahead: it looks up the signature of the current history, takes the predicted **next** stride, adds it to the running line address, shifts it into a temporary copy of the history and repeats, so the lookahead follows the learned pattern step by step and stops as soon as there is no entry of confidence ≥ 1 . Every candidate must stay inside the **same 4 KB page** as the demand access, since the next physical page is unknown; the first candidate that crosses the page ends the walk. The fill level depends on the miss-handling pressure: a prefetch fills the L1D when fewer than half of the MSHRs are occupied, and otherwise only the L2C, so prefetches do not starve demand misses.

Prefetch filter and usefulness tracking.

Each issued prefetch is recorded in a 64-entry direct-mapped filter (valid bit and 12-bit tag). It has two roles. First, a candidate already present in the filter is not requested again (duplicate suppression). Second, when a later demand load hits a line recorded in the filter, that prefetch is counted as *useful* and the entry is cleared. This gives the prefetcher its own accuracy, useful/issued, independent of ChampSim's counters.

Accuracy-based throttling.

Issued and useful prefetches are counted over an epoch of 256 issued prefetches. At the end of an epoch the accuracy is computed and the degree for the next epoch is chosen from three levels: a low degree (1) if accuracy is below 40 %, a high degree (4) if it is above 75 %, and a middle degree (2) otherwise; the counters are then reset. Thus the prefetcher is aggressive when its predictions are good and falls back when they are mostly wrong, which limits cache pollution and wasted bandwidth. The choice of the thresholds and degrees is discussed in Section 1.2.3.

Example.

Consider an IP with the stride sequence $+1, +3, +1, +3, \dots$, which IP-stride can never prefetch. Its uniform predictor keeps being contradicted, so `uni_conf` stays below 2 and the IP is non-uniform. The signature table, however, learns the two entries $(+1, +3, +1) \rightarrow +3$ and $(+3, +1, +3) \rightarrow +1$; after a few repetitions their confidence is ≥ 1 and, from the current history, SPP prefetches $+3, +1, +3, \dots$ lines ahead, up to the degree. A single irregular stride in a constant stream only lowers `uni_conf` by one, so the stream remains classified as uniform and keeps being prefetched.

1.2.2 Storage overhead

The storage is counted bit-exactly with the logical field widths of the hardware, not the C++ type sizes. Strides are 7-bit signed values (range ± 63) and a line address is $64 - 6 = 58$ bits. Table 3 lists every field.

Table	Field	Bits per entry	Entries	Total bits
<i>Signature entry (inside every tracker)</i>				
	tag	6		
	next stride	7		
	confidence	2		
	entry total	15	16 per tracker	240 per tracker
<i>IP tracker</i>				
	IP tag	8		
	last_line	58		
	uni_stride	7		
	uni_conf	2		
	hist[3] (3×7)	21		
	LRU rank	5		
	signature table (16×15)	240		
	tracker total	341	24	8184
<i>Prefetch filter</i>				
	valid bit	1		
	tag	12		
	entry total	13	64	832
<i>Throttle control</i>				
	epoch_issued (0–256)	9		
	epoch_useful (0–256)	9		
	degree (3 levels)	2		
	total	20	1	20
Total SPP storage				9036 bits
				(1129.5 bytes $\approx$ 1.10 KB)

Table 3: Storage overhead of SPP.

The trackers dominate the cost: $24 \times 341 = 8184$ bits, i.e. 90.6 % of the total, and about 70 % of each tracker is its private signature table (240 of 341 bits). The filter adds 832 bits and the control state only 20 bits. The statistics counters (`tot_issued` etc.) are only used for the final printout, are not part of the prefetcher logic, and are not counted. The same breakdown is printed by `l1d_prefetcher_final_stats` at the end of each run.

1.2.3 Accuracy-based throttling

Threshold selection.

The throttle works on epochs of $N = 256$ issued prefetches. At the end of an epoch the accuracy is $\text{acc} = 100 \cdot \text{useful}/\text{issued}$ (integer percent), computed from the prefetcher's own filter counters (Section 1.2), and the degree for the next epoch is

$$\text{degree} = \begin{cases} 1 & \text{acc} < T_{\text{low}} = 40\%, \\ 4 & \text{acc} > T_{\text{high}} = 75\%, \\ 2 & \text{otherwise.} \end{cases}$$

The counters are reset after every epoch, so the decision always reflects only the most recent 256 prefetches. The epoch is counted in issued prefetches rather than in cycles or accesses, so the accuracy estimate always has the same sample size, and 256 is large enough that one noisy prefetch moves the estimate by less than 0.4 percentage points.

The two thresholds split the accuracy range into three regions with a different cost of a wrong prefetch. Below 40 % most prefetches are wasted, and each wrong prefetch occupies an MSHR and L2/L1 bandwidth and can evict a useful line, so the prefetcher is throttled to the minimum. Above 75 % at least three of four prefetches are used, so going deeper is worth it. In between the effect of the degree is unclear and a moderate value is kept. The measured accuracy of IP-stride on our traces (53 %, 38 % and 16 %, Table 1) also lies around and below the lower threshold, which shows that a low-accuracy region needs to be handled explicitly.

[TODO: how the thresholds were decided: add the fixed-degree sweep and the $(T_{\text{low}}, T_{\text{high}})$ sweep (IPC, accuracy, cover

Choice of prefetch degrees.

The three levels are chosen to give the throttle a clear role for each accuracy region:

- **Degree 1 (accuracy < 40 %).** The lowest degree that still keeps the prefetcher running. It prefetches only the next predicted line, the one with the highest confidence, so the cost of wrong predictions is minimal, while the prefetcher can still recover when the program enters a regular phase. The prefetcher is not switched off completely, because then no new usefulness would be measured and the degree could never rise again.
- **Degree 2 (between 40 % and 75 %).** The default and the start value. It hides more latency than degree 1 while limiting the waste if some predictions are wrong.
- **Degree 4 (accuracy > 75 %).** For regular streams, where the lines farther ahead are almost always used. A deeper lookahead is needed to be timely, since the prefetch has to complete before the demand access arrives. Four is the maximum, because it is kept within the page limit (a 4KB page has only 64 lines) and because a large degree raises the pressure on the MSHRs and the prefetch filter (64 entries).

The degrees are in a 1 : 2 : 4 ratio so that each step is clearly different from the next.

[TODO: support the choice of 1/2/4 with the fixed-degree sweep (degree 1,2,3,4,6,8 with throttling off), e.g. where IPC

1.2.4 Results

[TODO: IPC, speedup, L1D load MPKI, accuracy, coverage: baseline vs IP-stride vs SPP]

Trace	Prefetcher	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage (%)
[TODO:]						

Table 4: Task 1 results.

[TODO: speedup plot]

Figure 3: Speedup over the no-prefetching baseline.

[TODO: L1D load MPKI plot]

Figure 4: L1D load MPKI.

[TODO: accuracy plot]

Figure 5: L1D prefetch accuracy.

[TODO: coverage plot]

Figure 6: L1D prefetch coverage.

1.2.5 Discussion

[TODO: where SPP wins or loses against IP-stride, and why]

2 Task 2: Cache Replacement Policy

2.1 Task 2A: Instruction-conscious L2 replacement (I-RRIP)

2.1.1 Design

[TODO: changes to `update_replacement_state`, insertion, promotion, eviction]

2.1.2 Rationale

[TODO: why insertion, promotion and eviction were modified or not modified, and why this choice is best]

2.1.3 Results

[TODO: IPC, speedup, Load MPKI of L2C, L2 Instruction and L2 Data]

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
[TODO:]						

Table 5: Task 2A results.

[TODO: plots]

2.2 Task 2B: Dynamic I-RRIP

2.2.1 Design

[TODO: set dueling, leader sets, decision formula]

2.2.2 Rationale for the leader sets and decision logic

[TODO: number of leader sets and why; decision logic and why]

2.2.3 Sensitivity study

[TODO: 5 values of the decision parameter (e.g. threshold) and their effect]

Parameter value	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
[TODO:]					

Table 6: Sensitivity of Dynamic I-RRIP to its decision parameter.

2.2.4 Results

[TODO: IPC, speedup, Load MPKI of L2C, L2 Instruction and L2 Data]

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
[TODO:]						

Table 7: Task 2B results.

[TODO: plots]

3 Task 3: Incorporating SPP with Dynamic I-RRIP

3.1 Design

[TODO: prefetch awareness in `replacement/dyn_irrip.l2c_repl`]

3.2 Rationale for handling prefetch packets

[TODO: why the replacement policy treats prefetch packets this way]

3.3 Results

[TODO: IPC, speedup, prefetch accuracy, coverage, Load MPKI of L1D, L2C, L2 Instruction and L2 Data]

Trace	Config	IPC	Speedup	Acc. (%)	Cov. (%)	L1D MPKI	L2C MPKI	L2 instr. MPKI	L2 Data MPKI
[TODO:]									

Table 8: Task 3 results: SPP + Dynamic I-RRIP versus SPP alone and Dynamic I-RRIP alone.

[TODO: plots]

3.4 Discussion

[TODO: comparison against standalone SPP and standalone Dynamic I-RRIP]